UNIVERSIDAD ANÁHUAC MAYAB

Ingeniería en Tecnologías de la Información y Negocios Digitales

Resistencia eléctrica

Proyecto de arreglos

Documentación del proyecto

Integrantes	Marco Antonio Gómez Fuentes
	Emiliano Peniche Correa
	Amir Moisés Hernández Farah
	Rodrigo Quijano López
Equipo	ProyectoFinal 7
Asignatura	Estructuras de datos
Lenguaje	Python
Fecha de entrega	20 de septiembre de 2026
Repositorio	https://github.com/MarcoGomez15/Portafolio-Anahuac-Mayab/tree/main/ProyectoResistencias

Fase 1. Contextualización

¿Qué se debe hacer?

Se debe hacer un programa que trabaje con resistencias eléctricas y resuelva dos cosas: traducir el valor de una resistencia en ohmios a los colores de sus bandas, y calcular el valor total de n resistencias conectadas en serie y en paralelo. En concreto el programa debe:

- Recibir el valor de una resistencia y dar los colores de la primera banda (primer dígito), la segunda (segundo dígito) y la tercera (multiplicador), mostrándolos además de forma gráfica.
- Pedir al usuario la cantidad n de resistencias y darle a cada una un valor entero aleatorio entre 10 y 1 000 000 000 ohmios, guardándolas en un arreglo.
- Calcular el total en serie con Rts = R1 + R2 + … + Rn.
- Calcular el total en paralelo con 1/Rtp = 1/R1 + 1/R2 + … + 1/Rn.

¿Qué se necesita?

- La tabla del código de colores de 4 bandas, que dice qué dígito, multiplicador y tolerancia vale cada color.
- El procedimiento inverso a leer una resistencia: separar el valor en dos cifras significativas y una potencia de diez.
- Las fórmulas de la resistencia total en serie y en paralelo.
- Un arreglo para guardar las n resistencias y recorrerlo para acumular el total.
- Un generador de números aleatorios dentro del rango permitido.
- Python 3, que se ejecuta desde la terminal y no necesita instalar bibliotecas.

¿Qué elementos se identifican?

Del enunciado salen los siguientes elementos, que después se convirtieron en las clases del programa:

Elemento	Tipo	Descripción
Color de banda	Objeto	Cada uno de los 13 colores, con su dígito, su multiplicador y su tolerancia.
Tabla del código de colores	Arreglo de objetos	Los 13 colores juntos; es donde el programa consulta.
Resistencia	Objeto	Componente con un valor en ohmios del que salen las cuatro bandas.
Arreglo de resistencias	Arreglo de objetos	Conjunto de n resistencias generadas al azar.
Valor total en serie	Dato calculado	Suma de los valores del arreglo (Rts).
Valor total en paralelo	Dato calculado	Inverso de la suma de los inversos (Rtp).
Menú de consola	Objeto	Pide los datos, dibuja la resistencia y muestra los resultados.

Fase 2. Definición de los requerimientos funcionales

Listado de requerimientos que salen del contexto planteado:

Clave	Requerimiento	Origen
R1	Determinar los colores de una resistencia a partir de su valor	Requerimiento 1 del enunciado
R2	Calcular el valor total de n resistencias en serie	Requerimiento 2 del enunciado
R3	Calcular el valor total de n resistencias en paralelo	Requerimiento 3 del enunciado
R4	Generar n resistencias con valores enteros aleatorios	Condición de R2 y R3
R5	Mostrar la resistencia y sus colores en modo gráfico	Condición de la entrega en ordinario
R6	Validar los datos que escribe el usuario	Buenas prácticas

R1 – Determinar los colores de una resistencia

Nombre	R1 – Determinar los colores de una resistencia
Descripción	A partir del valor en ohmios el sistema calcula el primer dígito, el segundo dígito y el exponente del multiplicador, y busca en la tabla el color de cada banda. La cuarta banda es la tolerancia que elige el usuario.
Entrada	Un número entero entre 10 y 1 000 000 000, y el color de tolerancia (Dorado, Plateado o Ninguno).
Salidas	Los nombres de los colores de las tres bandas de valor. Para 160 da Marrón, Azul, Marrón; para 5800 da Verde, Gris, Rojo.

R2 – Calcular el valor total en serie

Nombre	R2 – Calcular el valor total en serie
Descripción	El sistema recorre el arreglo y suma los valores de todas las resistencias, aplicando Rts = R1 + R2 + … + Rn.
Entrada	El arreglo de n resistencias generado en R4.
Salidas	El valor total Rts en ohmios, con su equivalencia en kΩ, MΩ o GΩ.

R3 – Calcular el valor total en paralelo

Nombre	R3 – Calcular el valor total en paralelo
Descripción	El sistema recorre el arreglo sumando el inverso de cada resistencia y devuelve el inverso de esa suma, según 1/Rtp = 1/R1 + 1/R2 + … + 1/Rn.

Entrada	El arreglo de n resistencias generado en R4.
Salidas	El valor total Rtp en ohmios, con su equivalencia en kΩ, MΩ o GΩ.

R4 – Generar n resistencias aleatorias

Nombre	R4 – Generar n resistencias aleatorias
Descripción	El usuario dice cuántas resistencias quiere y el sistema crea n objetos Resistencia con un valor entero al azar dentro del rango, guardándolos en el arreglo. El mismo arreglo se usa para serie y para paralelo, para que los dos resultados se puedan comparar.
Entrada	Un número entero n mayor o igual a 1.
Salidas	Un arreglo con n resistencias, mostrado en una tabla con su valor, su equivalencia y sus colores.

R5 – Mostrar la resistencia en modo gráfico

Nombre	R5 – Mostrar la resistencia en modo gráfico
Descripción	El sistema dibuja la resistencia en la terminal con bloques de texto: el cuerpo, los alambres y las cuatro bandas pintadas con su color real, y debajo de cada banda escribe qué representa y de qué color es.
Entrada	El objeto Resistencia con sus bandas ya calculadas.
Salidas	El dibujo de la resistencia con sus bandas de color.

R6 – Validar los datos capturados

Nombre	R6 – Validar los datos capturados
Descripción	Antes de calcular, el sistema revisa que el valor sea un número entero entre 10 y 1 000 000 000 y que n sea un entero mayor o igual a 1. Si el dato no sirve, muestra un aviso y lo vuelve a pedir; con la letra c se cancela y se regresa al menú.
Entrada	Lo que escribe el usuario en la terminal.
Salidas	Un mensaje de advertencia, o la continuación normal del proceso.

Fase 3. Diseño

El sistema quedó en ocho clases y un módulo de funciones. Tres clases resuelven el código de colores y la resistencia, tres forman la jerarquía de arreglos y dos se encargan de la presentación en la terminal. La herencia está entre ArregloResistencias y sus dos clases hijas: la clase padre guarda el arreglo, lo llena con valores aleatorios y da el formato del resultado, y las hijas solo cambian calcular_total() y obtener_formula() con la fórmula de cada tipo de conexión.

El diagrama de clases se presenta en tres partes para que se pueda leer bien:

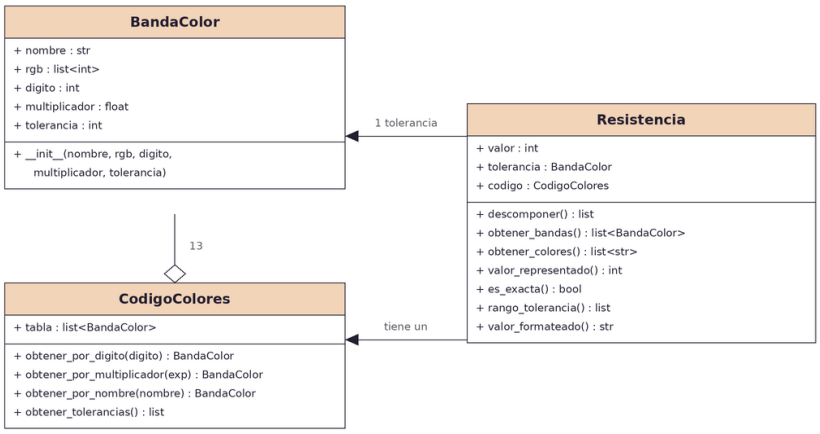


Figura 1. Clases del código de colores y de la resistencia.

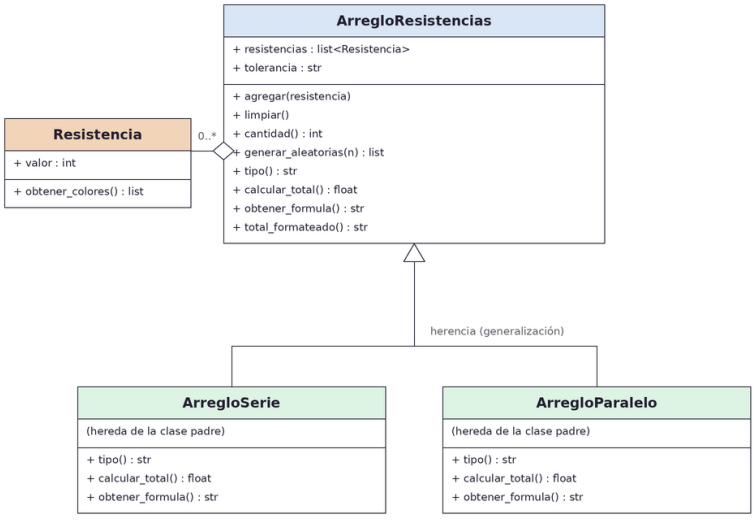


Figura 2. Jerarquía de arreglos, donde está la relación de herencia.

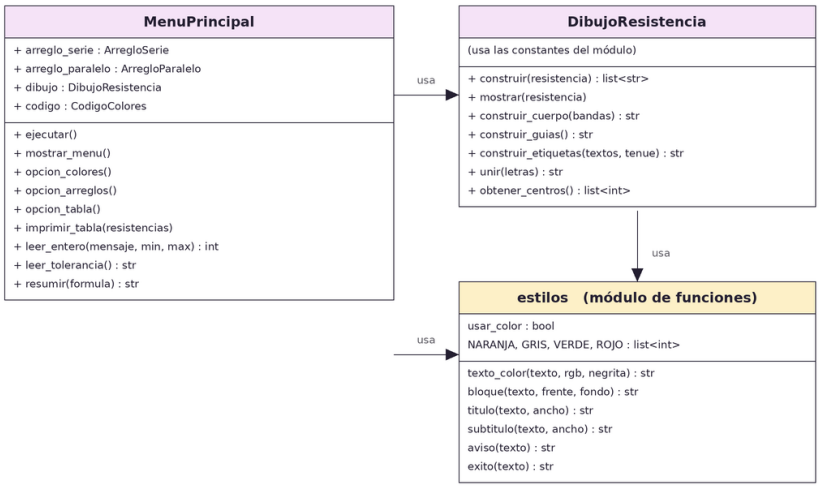


Figura 3. Clases de la interfaz de consola.

Clases del sistema

Clase	Atributos	Métodos
BandaColor	nombre, rgb, digito, multiplicador, tolerancia	constructor
CodigoColores	tabla (arreglo de BandaColor)	obtener_por_digito, obtener_por_multiplicador, obtener_por_nombre, obtener_tolerancias
Resistencia	valor, tolerancia, codigo	descomponer, obtener_bandas, obtener_colores, valor_representado, es_exacta, rango_tolerancia, valor_formateado
ArregloResistencias	resistencias (arreglo), tolerancia	agregar, limpiar, cantidad, generar_aleatorias, tipo, calcular_total, obtener_formula, total_formateado
ArregloSerie	hereda los de la clase padre	tipo, calcular_total, obtener_formula
ArregloParalelo	hereda los de la clase padre	tipo, calcular_total, obtener_formula
DibujoResistencia	constantes del dibujo	construir, mostrar, construir_cuerpo, construir_guias, construir_etiquetas, unir, obtener_centros
MenuPrincipal	arreglo_serie, arreglo_paralelo, dibujo, codigo	ejecutar, mostrar_menu, opcion_colores, opcion_arreglos, opcion_tabla, imprimir_tabla, leer_entero, leer_tolerancia, resumir

Relaciones

Relación	Tipo	Descripción
ArregloSerie → ArregloResistencias	Herencia	Cambia calcular_total() por la suma directa.

Relación	Tipo	Descripción
ArregloParalelo → ArregloResistencias	Herencia	Cambia calcular_total() por el inverso de la suma de inversos.
ArregloResistencias ◇ Resistencia	Agregación	Un arreglo guarda de 0 a n resistencias.
CodigoColores ◇ BandaColor	Agregación	La tabla está formada por 13 colores.
Resistencia → CodigoColores	Asociación	Consulta la tabla para obtener sus bandas.

Fase 4. Codificación

El programa se escribió en Python 3 siguiendo el diagrama de la fase anterior. Los archivos son:

Archivo	Contenido
main.py	Punto de entrada; crea el menú y lo pone a correr.
codigo_colores.py	Clases BandaColor y CodigoColores.
resistencia.py	Clase Resistencia y las funciones de apoyo (R1 y R4).
arreglo_resistencias.py	Clase padre del arreglo de resistencias.
arreglo_serie.py y arreglo_paralelo.py	Clases hijas con las fórmulas (R2 y R3).
estilos.py	Funciones que aplican el color de la terminal.
dibujo_resistencia.py	Dibuja la resistencia y sus bandas (R5).
menu_principal.py	Menú y validación de los datos (R6).
pruebas.py	Programa de pruebas de la fase 5.

Lo más importante del R1 es separar el valor en dos cifras significativas y el exponente del multiplicador, que es justo lo que codifican las tres primeras bandas:

resistencia.py

```
# Separa el valor en sus 2 primeros dígitos y el exponente del multiplicador
# ej. 5800 -> [5, 8, 2] porque 58 x 10^2 = 5800
def descomponer(self):
    exponente = len(str(self.valor)) - 2
    base = 10 ** exponente
    mantisa = (self.valor + base // 2) // base

    # si al redondear sale 100 (ej. 999) lo paso a 10 y subo el exponente
    if mantisa == 100:
        mantisa = 10
        exponente = exponente + 1

    primer_digito = mantisa // 10
    segundo_digito = mantisa % 10
    return [primer_digito, segundo_digito, exponente]
```

Cada clase hija implementa su fórmula recorriendo el arreglo de resistencias:

arreglo_serie.py

```
# Suma todas las resistencias del arreglo: Rts = R1 + R2 + ... + Rn
def calcular_total(self):
    total = 0.0
    for i in range(len(self.resistencias)):
        total = total + self.resistencias[i].valor
    return total
```

arreglo_paralelo.py

```
# Suma los inversos de cada resistencia y el total es el inverso de esa suma
# 1/Rtp = 1/R1 + 1/R2 + ... + 1/Rn
def calcular_total(self):
    if len(self.resistencias) == 0:
        return 0.0

    suma_inversos = 0.0
    for i in range(len(self.resistencias)):
        suma_inversos = suma_inversos + 1 / self.resistencias[i].valor
    return 1 / suma_inversos
```

Todas las clases y todos los métodos llevan un comentario que explica qué hacen. El programa usa solo clases, herencia, arreglos, ciclos, condicionales y funciones, con nombres en español y con la lógica separada de la interfaz.

Fase 5. Pruebas

Se probó cada requerimiento con los casos del enunciado y con los valores límite del rango:

#	Req.	Entrada	Salida esperada	Salida obtenida	Resultado
1	R1	160 Ω	Marrón, Azul, Marrón	Marrón, Azul, Marrón	Correcto
2	R1	5800 Ω	Verde, Gris, Rojo	Verde, Gris, Rojo	Correcto
3	R1	10 Ω (mínimo)	Marrón, Negro, Negro	Marrón, Negro, Negro	Correcto
4	R1	1 000 000 000 Ω (máximo)	Marrón, Negro, Gris	Marrón, Negro, Gris	Correcto
5	R2	100, 220 y 330 Ω en serie	650 Ω	650 Ω	Correcto
6	R3	100, 220 y 330 Ω en paralelo	56.8966 Ω	56.8966 Ω	Correcto
7	R3	1000 y 1000 Ω en paralelo	500 Ω	500 Ω	Correcto
8	R4	n = 8	8 resistencias dentro del rango	8 resistencias dentro del rango	Correcto
9	R5	Resistencia de 160 Ω	Dibujo con 4 bandas de color	Dibujo con 4 bandas de color	Correcto
10	R6	9 Ω, el texto abc y n = 0	Mensaje de advertencia	Mensaje de advertencia	Correcto

Estos casos también se automatizaron en el archivo pruebas.py, que se corre con el comando python pruebas.py y termina con el resultado "Pruebas correctas: 32 de 32".

Evidencias de la ejecución

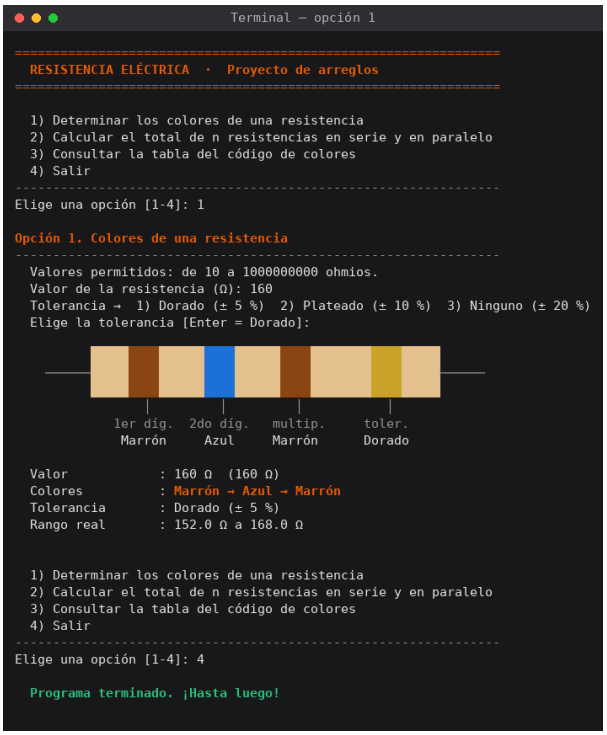


Figura 4. R1 y R5: resistencia de 160 Ω (Marrón, Azul, Marrón).



Figura 5. R1: resistencia de 5 800 Ω (Verde, Gris, Rojo) con tolerancia Plateado.

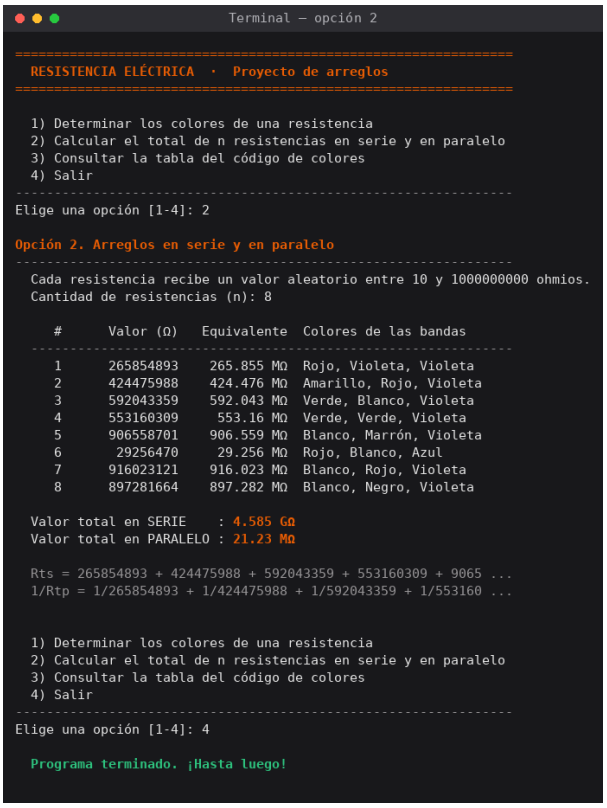


Figura 6. R2, R3 y R4: 8 resistencias aleatorias con su total en serie y en paralelo.



Figura 7. R6: el programa rechaza el texto abc, el valor 9 Ω y n = 0, y vuelve a pedir el dato.

Conclusión

El programa cumple con los tres requerimientos del enunciado y con los que se derivaron de ellos. Los colores se comprobaron con los dos ejemplos del proyecto y con los valores límite, y los totales en serie y en paralelo con valores conocidos. Usar herencia permitió escribir una sola vez la administración del arreglo, de modo que agregar otro tipo de conexión en el futuro solo requeriría una clase hija más.

Referencias

Hernández Bejarano, M. y Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica (1 ed.). Madrid, RA-MA Editorial. Recuperado de https://elibro.net/es/ereader/mayab/230581